API Security Basics

Sample knowledge document for testing SentinelAI PDF ingestion and retrieval.

1. Authentication

An API should verify who is making a request before allowing access to protected resources. Common approaches include API keys for service-to-service integrations and short-lived access tokens for user sessions. Store secrets in environment variables or a secret manager, not in source code.

2. Input Validation

Validate incoming data at the API boundary. Check required fields, data types, string lengths, and allowed file formats. Reject malformed requests with a clear client error instead of passing untrusted input deeper into the application.

3. Rate Limiting

Rate limiting restricts how many requests a client can make during a time window. It helps protect against accidental overload and some forms of abuse. Limits can be applied per API key, user, or IP address.

4. Error Handling

Return useful status codes and safe error messages. Do not expose stack traces, database credentials, internal file paths, or secret values in API responses. Keep diagnostic details in protected server-side logs.

5. File Upload Safety

For uploaded files, enforce a size limit, allow only expected file types, generate safe destination names, and do not trust filenames supplied by clients. Consider malware scanning and store uploads outside publicly served directories.

6. HTTPS

HTTPS encrypts data in transit between clients and servers. Production APIs should use HTTPS to reduce interception or tampering risks, especially when requests contain credentials or personal information.

Quick review questions

Why should secrets not be hardcoded? What is the purpose of rate limiting? Name two checks to perform on uploaded files. Why should detailed stack traces stay out of API responses?